

Laporan Tugas Besar 4

II4021 - Kriptografi



Disusun oleh:

Benedict Presley	13523067
Nayaka Ghana Subrata	13523090
Luckman Fakhmanidris Arvasirri	18223041

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2025/2026

DAFTAR ISI

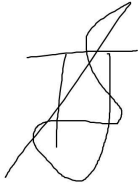
Pernyataan Tidak Melakukan Kecurangan.....	4
I. TEORI.....	5
1. Shamir Secret Sharing Scheme (SSS).....	5
2. AES-GCM (Advanced Encryption Standard – Galois/Counter Mode).....	6
3. Key Derivation Function (KDF).....	7
4. CSPRNG (Cryptographically Secure Pseudorandom Number Generator).....	8
5. Kriptografi Visual.....	9
II. PERANCANGAN DAN IMPLEMENTASI.....	11
2.1 Pembuatan Vault dan Skema Pembagian Rahasia.....	11
2.1.1 Skema Shamir Secret Sharing.....	11
2.1.2 Fungsi Derivasi Kunci (KDF) dan Enkripsi Local Share.....	11
2.2 Alur Akses dan Pemulihan Vault.....	11
2.3 Sistem Pembangkit Password Otomatis (CSPRNG).....	12
2.4 Kriptografi Visual untuk Recover Share (Bonus).....	12
III. PENGUJIAN DAN ANALISIS.....	14
3.1 Uji Pembuatan Vault.....	14
3.2 Uji Penyimpanan dan Perlindungan local share.....	17
3.3 Uji Akses Normal.....	20
3.4 Uji Penambahan Data Password.....	24
3.5 Uji Pengubahan dan Penghapusan Data Password.....	26
3.6 Uji Penyimpanan Vault di Server.....	29
3.7 Uji Mode Backup.....	31
3.8 Uji Kegagalan Pemulihan.....	36
3.9 Uji Kriptografi Visual.....	40
IV. KESIMPULAN.....	47
DAFTAR PUSTAKA.....	49
LAMPIRAN.....	50

DAFTAR GAMBAR

Gambar 1.1 Ilustrasi Shamir Secret Sharing (SSS).....	5
Gambar 1.2 Arsitektur AES-GCM.....	6
Gambar 1.3 Key Derivation Function (KDF) Argon 2.....	7
Gambar 1.4 Salah Satu Contoh Algoritma CSPRNG yang Sering Digunakan (ChaCha20).....	9
Gambar 1.5 Contoh Cara Kerja Visual Cryptography.....	9
Gambar 2.1 Diagram Alur Akses dan Pemulihan Vault.....	12
Gambar 3.1-a Proses Pembuatan Vault Baru.....	16
Gambar 3.1-b Key yang dihasilkan.....	17
Gambar 3.1-c Recovery share.....	17
Gambar 3.1-d Data yang disimpan server.....	17
Gambar 3.2-a Data local share yang tersimpan di sisi klien.....	19
Gambar 3.2-b local share dienkripsi dari turunan password.....	19
Gambar 3.2-c Local share berhasil didekripsi dari master password yang benar.....	19
Gambar 3.2-c Local share gagal didekripsi dari master password yang salah.....	20
Gambar 3.3-a Penggunaan local share + server share pada sistem.....	22
Gambar 3.3-b Master key berhasil direkonstruksi.....	23
Gambar 3.3-c Vault berhasil didekripsi.....	23
Gambar 3.4-c Vault gagal didekripsi.....	24
Gambar 3.7-a Simulasi kondisi server tidak dapat diakses.....	32
Gambar 3.7-b Kombinasi local share dan recovery share pada sistem.....	34
Gambar 3.7-c Backup vault terenkripsi.....	35
Gambar 3.7-d Password dapat dilihat pada backup mode.....	36
Gambar 3.7-e Hanya bisa melihat entry pada backup mode.....	36
Gambar 3.8-a Kegagalan pemulihan akibat recovery share tidak valid.....	38
Gambar 3.8-b Kegagalan dekripsi akibat backup vault tidak sesuai.....	39
Gambar 3.8-c Sistem tidak menampilkan isi vault ketika pemulihan gagal.....	40
Gambar 3.9-a QR code awal dari recovery share.....	42
Gambar 3.9-b Dua visual share hasil pembagian QR code.....	44
Gambar 3.9-c Hasil penggabungan dua visual share.....	45
Gambar 3.9-d Recovery share hasil pemindaian QR rekonstruksi.....	46

Pernyataan Tidak Melakukan Kecurangan

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.



Benedict Presley



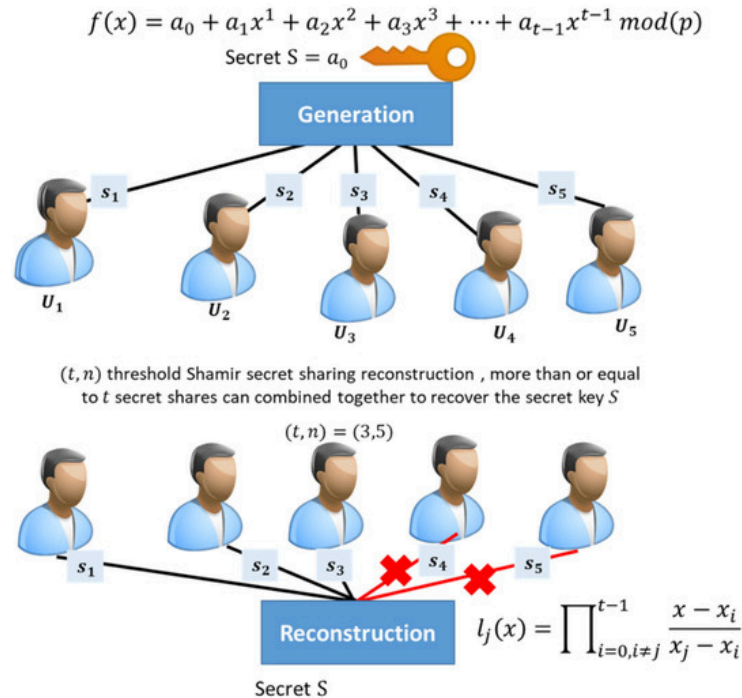
Nayaka Ghana Subrata



Luckman F. Arvasirri

I. TEORI

1. Shamir Secret Sharing Scheme (SSS)



Gambar 1.1 Ilustrasi Shamir Secret Sharing (SSS)

Sumber:

https://www.mdpi.com/sensors/sensors-22-00331/article_deploy/html/images/sensors-22-00331-g001-550.jpg

Shamir Secret Sharing adalah skema kriptografi yang dikembangkan oleh Adi Shamir pada tahun 1979. Skema ini memungkinkan sebuah secret dibagi menjadi n bagian (share), sehingga hanya diperlukan minimal k share untuk merekonstruksi secret tersebut. Skema ini dinotasikan sebagai skema ambang batas (threshold scheme) (k, n) .

SSS dibangun di atas fakta bahwa sebuah polinomial berderajat $k - 1$ dapat ditentukan secara unik dari tepat k titik. Misalkan secret yang ingin dibagi adalah s . Dibuat polinomial berderajat $k - 1$ sebagai berikut:

$$f(x) = s + a_1x + a_2x^2 + \dots + a_{k-1}x^{k-1} \text{ (mod } p)$$

di mana $a_1, a_2, \dots, a_{k-1}$ adalah koefisien yang dibangkitkan secara acak, dan p adalah bilangan prima yang lebih besar dari s maupun n . Secret s adalah nilai $f(0)$, yaitu konstanta polinomial.

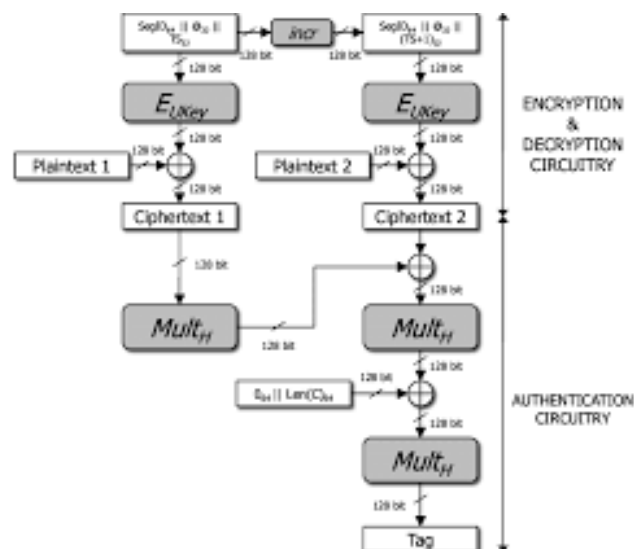
Setiap share ke- i adalah pasangan $(i, f(i))$ untuk $i = 1, 2, \dots, n$. Rekonstruksi dilakukan menggunakan **Interpolasi Lagrange** untuk menemukan nilai $f(0)$:

$$f(0) = \sum_i^k y_i \prod_{j=1, j \neq i}^k \frac{-x_j}{x_i - x_j}$$

Seluruh operasi aritmatika dilakukan dalam finite field $GF(p)$ (aritmatika modulo prima) agar semua nilai tetap terbatas dan hasil rekonstruksi selalu tepat.

SSS memiliki sifat perfect secrecy. Kepemilikan kurang dari k share tidak memberikan informasi apapun tentang nilai secret. Selain itu, skema ini toleran terhadap kehilangan share. Selama minimal k share masih tersedia, secret tetap dapat direkonstruksi.

2. AES-GCM (Advanced Encryption Standard – Galois/Counter Mode)



Gambar 1.2 Arsitektur AES-GCM

Sumber:

https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQOoEljOvuD8M7QC62JI_Z97Prp9B5ayJs2Q&s

AES adalah algoritma enkripsi simetris yang ditetapkan oleh NIST pada tahun 2001 sebagai standar enkripsi federal Amerika Serikat. AES beroperasi pada blok data 128-bit dengan ukuran kunci 128, 192, atau 256 bit.

GCM (Galois/Counter Mode) adalah mode operasi yang mengkombinasikan CTR mode untuk enkripsi dengan autentikasi berbasis perkalian di Galois field $GF(2^{128})$. Kombinasi ini menjadikan AES-GCM sebagai skema AEAD (Authenticated Encryption with Associated Data), yang memberikan dua jaminan sekaligus:

- Kerahasiaan (confidentiality): Plaintext dienkripsi menggunakan CTR mode sehingga tidak dapat dibaca tanpa kunci.
- Integritas dan autentikasi (integrity & authenticity): Tag autentikasi 128-bit dihasilkan dari ciphertext. Jika ciphertext dimodifikasi sekecil apapun, verifikasi tag akan gagal dan data ditolak.

AES-GCM terdiri dari 3 komponen:

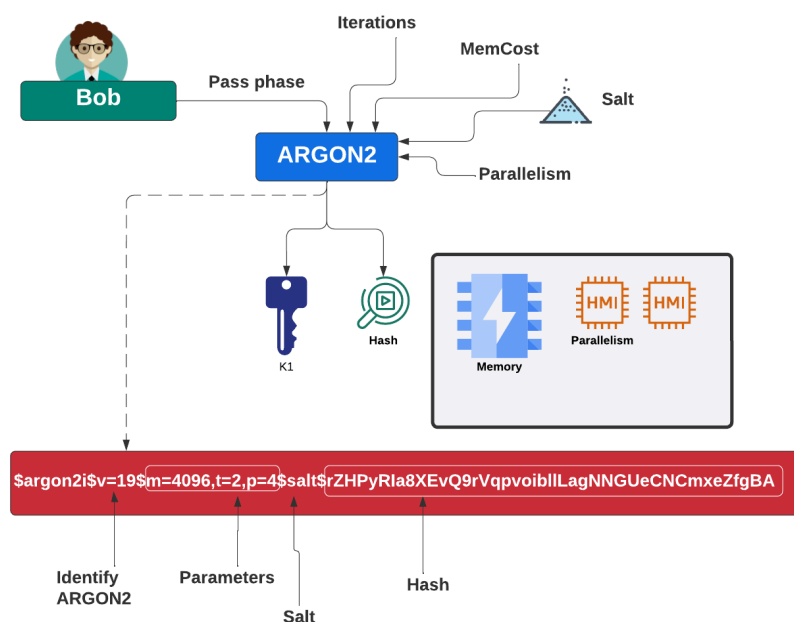
- Key: bisa berupa 128/192/256 bit. Bertujuan sebagai kunci untuk enkripsi.
- Nonce (IV): Number used once. Harus unik per operasi enkripsi dengan kunci yang sama.
- Auth Tag: Digunakan untuk verifikasi integritas ciphertext.

Nonce harus benar-benar unik untuk setiap operasi enkripsi yang menggunakan kunci yang sama. Penggunaan nonce yang sama dua kali (nonce reuse) pada GCM merupakan kerentanan kritis karena penyerang dapat memulihkan kunci autentikasi dan memalsukan ciphertext. Cara yang aman adalah membangkitkan nonce secara acak menggunakan CSPRNG setiap kali enkripsi dilakukan.

3. Key Derivation Function (KDF)

Key Derivation Function adalah fungsi yang menurunkan satu atau lebih kunci kriptografis dari sebuah nilai sumber seperti password atau shared secret. KDF diperlukan karena password manusia umumnya memiliki entropi rendah dan panjang yang bervariasi, sehingga tidak dapat langsung digunakan sebagai kunci kriptografis yang berukuran tetap.

KDF yang dirancang khusus untuk menurunkan kunci dari password (password-based KDF) memiliki sifat tambahan: dirancang lambat secara komputasi dan membutuhkan sumber daya besar, sehingga serangan brute-force terhadap password menjadi tidak praktis.



Gambar 1.3 Key Derivation Function (KDF) Argon 2

Sumber:

https://miro.medium.com/v2/resize:fit:1400/1*_nLbiLzewGDJ7oYMH-D6fw.png

Argon2 adalah algoritma KDF pemenang Password Hashing Competition (PHC) tahun 2015 dan direkomendasikan oleh OWASP serta RFC 9106. Argon2 memiliki tiga varian:

- Argon2d: Tahan terhadap serangan GPU, namun rentan terhadap side-channel attack.
- Argon2i: Tahan terhadap side-channel attack, namun kurang tahan terhadap serangan GPU.
- Argon2id: Varian hybrid yang menggabungkan ketahanan keduanya.

Argon2id bersifat memory-hard, artinya komputasinya memerlukan sejumlah besar memori RAM. Hal ini membuat serangan brute-force menggunakan GPU atau ASIC menjadi sangat mahal secara sumber daya, karena jumlah percobaan paralel dibatasi oleh kapasitas memori perangkat penyerang.

Parameter utama Argon2id yang dapat dikonfigurasi adalah `time_cost` (jumlah iterasi), `memory_cost` (konsumsi memori dalam KB), `parallelism` (jumlah thread), dan `hash_len` (panjang output dalam byte). Selain itu, digunakan salt acak yang unik per pengguna untuk mencegah serangan rainbow table.

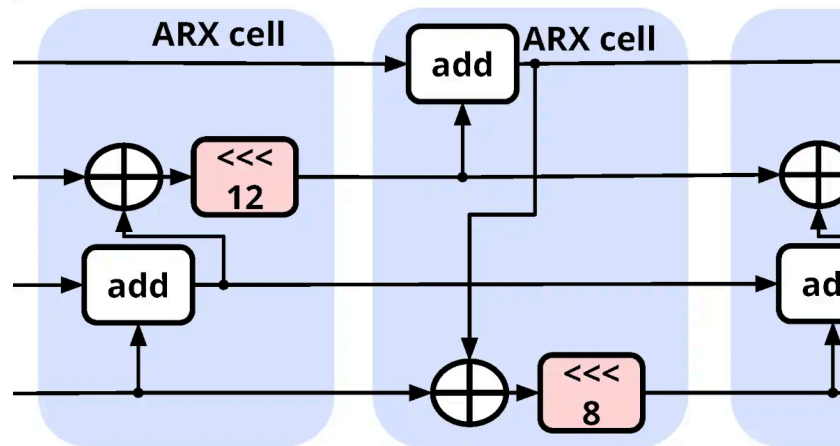
4. CSPRNG (Cryptographically Secure Pseudorandom Number Generator)

CSPRNG adalah generator bilangan acak yang memenuhi syarat keamanan kriptografis. Berbeda dari PRNG biasa yang hanya memenuhi syarat statistik, CSPRNG harus memenuhi dua syarat tambahan:

- Forward unpredictability: Mengetahui sejumlah output sebelumnya tidak memungkinkan prediksi output berikutnya.
- Backward unpredictability: Mengetahui state saat ini tidak memungkinkan rekonstruksi output sebelumnya.

PRNG seperti Mersenne Twister yang digunakan oleh banyak bahasa pemrograman secara default tidak aman secara kriptografis. State internal Mersenne Twister (19.937 bit) dapat sepenuhnya direkonstruksi dari 624 output 32-bit berturut-turut, sehingga seluruh output masa lalu dan masa depan dapat diprediksi.

CSPRNG menggunakan sumber entropi dari sistem operasi yang terus diperbarui dari kejadian fisik tak terduga seperti interrupt perangkat keras, timing operasi disk, dan thermal noise.



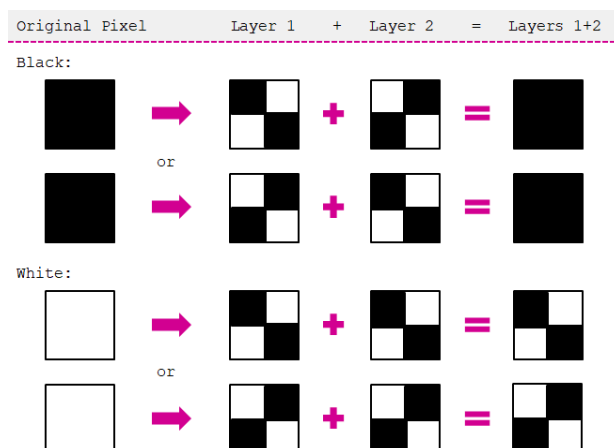
Gambar 1.4 Salah Satu Contoh Algoritma CSPRNG yang Sering Digunakan (ChaCha20)

Sumber:

<https://networkencyclopedia.com/wp-content/uploads/2023/08/chacha20.webp>

Untuk membangkitkan karakter acak dari sebuah charset, output CSPRNG dipetakan ke indeks charset menggunakan teknik rejection sampling atau modular reduction. Teknik rejection sampling lebih disukai karena menghasilkan distribusi seragam (uniform distribution) tanpa bias, sedangkan modular reduction dapat menghasilkan bias kecil jika ukuran charset tidak membagi habis rentang output CSPRNG.

5. Kriptografi Visual



Gambar 1.5 Contoh Cara Kerja Visual Cryptography

Sumber:

<https://www.101computing.net/wp/wp-content/uploads/visual-cryptography.png>

Kriptografi visual (*visual cryptography*) adalah skema kriptografi yang diperkenalkan oleh Naor dan Shamir pada tahun 1994. Skema ini memungkinkan sebuah gambar rahasia dibagi menjadi beberapa share berupa gambar, sehingga gambar asli hanya dapat direkonstruksi dengan melapiskan (overlay) sejumlah share.

Setiap share secara individual terlihat seperti gambar noise acak dan tidak mengungkapkan informasi apapun tentang gambar asli. Rekonstruksi dilakukan secara visual menggunakan operasi OR (atau AND, bergantung implementasi) pada piksel kedua share, tanpa memerlukan kunci atau komputasi tambahan.

II. PERANCANGAN DAN IMPLEMENTASI

2.1 Pembuatan Vault dan Skema Pembagian Rahasia

Alur pembuatan vault diinisiasi di sisi klien ketika pengguna memasukkan master password pertama kali. Klien akan membangkitkan kunci utama (master key) sepanjang 16 byte secara acak menggunakan fungsi murni `os.urandom` pada file `aes_gcm.py`. Struktur data minimal akun yang disimpan mencakup atribut nama layanan, username atau email, password, dan catatan opsional yang direpresentasikan dalam pada file `models.py`. Objek vault kosong diserialisasi menjadi biner melalui file `codec.py` dan langsung dienkripsi menggunakan algoritma AES-128-GCM dengan parameter nonce sepanjang 12 byte. Setelah itu, kunci utama dipecah menjadi tiga bagian share menggunakan skema Shamir Secret Sharing.

2.1.1 Skema Shamir Secret Sharing

Shamir Secret Sharing diimplementasikan dalam file `shamir.py` menggunakan skema ambang batas (2, 3). Operasi matematis pemecahan komponen kunci dihitung secara modular terhadap bilangan prima 257 dengan membangkitkan nilai *slope* (kemiringan) polinomial derajat satu secara acak. Proses membuahkn tiga bagian rahasia yaitu server share yang dikirim melalui `server_client.py` untuk disimpan di server bersama vault terenkripsi, recovery share dalam format teks koordinat (Base64 encoded) yang ditampilkan sekali kepada pengguna sebagai cadangan mandiri, dan local share yang dienkripsi secara lokal.

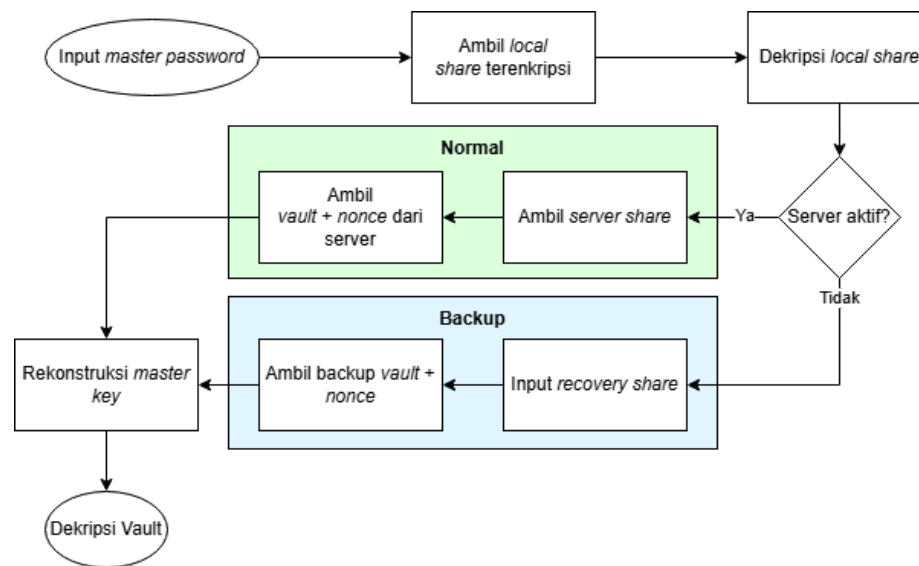
2.1.2 Fungsi Derivasi Kunci (KDF) dan Enkripsi Local Share

Algoritma Key Derivation Function (KDF) yang diimplementasikan adalah Argon2id dalam file `kdf.py`. Parameter KDF dikonfigurasi menggunakan salt acak sepanjang 16 bytes, 3 iterasi, memory cost sebesar 64 MB, dan paralelisme sebanyak 4 lanes untuk menghasilkan kunci berukuran 16 bytes. Kunci turunan tersebut digunakan untuk mengenkripsi plaintext, lalu hasil enkripsi beserta parameter local nonce dan salt disimpan ke dalam local storage berformat JSON melalui `ClientStorage` pada `storage.py`.

2.2 Alur Akses dan Pemulihan Vault

Logika operasional klien untuk membuka dan mengaktifkan akses vault dipisahkan menjadi dua mode akses seperti yang dapat dilihat pada file `manager.py`.

Pertama adalah Mode Normal yang digunakan sebagai akses utama ketika server aktif. Klien memanggil fungsi pada `server_client.py` untuk mengambil server share, encrypted vault, dan nonce dari server. Klien mendekripsi local share menggunakan kunci hasil derivasi master password, lalu menggabungkannya dengan server share menggunakan kombinasi share pada `shamir.py` untuk merekonstruksi master key. Setelah rekonstruksi berhasil, master key digunakan untuk mendekripsi vault agar pengguna dapat melihat, menambah, mengubah, dan menghapus data password. Setiap terjadi perubahan data, seluruh isi vault dienkripsi ulang sebagai satu kesatuan dengan nonce baru, lalu disimpan kembali ke server dan memperbarui data backup lokal.



Gambar 2.1 Diagram Alur Akses dan Pemulihan Vault

Sumber:

[Spesifikasi Tugas4](#)

Kedua adalah Mode Backup yang digunakan sebagai akses darurat ketika server tidak dapat diakses. Klien tidak melakukan request ke server, melainkan memuat local share payload, encrypted backup vault, dan backup nonce langsung dari `storage.py` sehingga pengguna wajib memasukkan teks recovery share secara manual. Klien menggabungkan local share dan recovery share untuk merekonstruksi master key guna mendekripsi backup vault lokal. Mode ini bersifat *read-only* dan tidak melakukan pembaruan data untuk mencegah terjadinya konflik versi antara backup lokal dengan vault di server.

2.3 Sistem Pembangkit Password Otomatis (CSPRNG)

Fitur pembangkitan password otomatis dikembangkan dalam file `cspng.py` menggunakan *Cryptographically Secure Pseudorandom Number Generator* (CSPRNG). Sistem menggunakan arsitektur stream cipher ChaCha20 dengan parameter key 32 bytes and nonce 16 bytes acak murni dari `os.urandom` untuk menghasilkan keystream. Guna memetakan bytes acak tersebut menjadi karakter password yang terdiri atas kombinasi huruf besar, huruf kecil, angka, dan simbol, diterapkan teknik rejection sampling dengan menggunakan threshold tertentu untuk menghindari adanya modulo bias pada distribusi karakter password.

2.4 Kriptografi Visual untuk Recover Share (Bonus)

Fitur *recovery share* dapat diimplementasikan menggunakan kriptografi visual dalam file `visual_crypto.py`. Teks *recovery share* terlebih dahulu diubah menjadi QR code dengan tingkat error correction M menggunakan library `qrcode`. QR code tersebut kemudian dibagi menggunakan skema visual cryptography (2, 2) menjadi dua buah image share berformat PNG. Setiap pixel pada matriks QR code diekspansi menggunakan sub-pixel pattern berukuran 2x2. Jika cell bernilai hitam, sub-pixel pattern pada share pertama dan kedua dibuat saling berkebalikan (invert), sedangkan untuk cell putih dibuat menggunakan pattern yang identik. Pada proses pemulihan darurat melalui CLI (file `app.py`), pengguna melakukan upload dengan memasukkan absolute file path dari

kedua image share tersebut. Klien melakukan operasi tumpukan menggunakan fungsi `ImageChops.darker` dari library Pillow untuk merekonstruksi visual overlay dari QR code. Hasil gabungan kedua image share tersebut kemudian dipindai kembali menggunakan modul `cv2.QRCodeDetector` dari OpenCV untuk decode payload teks *recovery share* yang sama secara presisi.

III. PENGUJIAN DAN ANALISIS

Bab ini membahas pengujian program Vault untuk memastikan bahwa seluruh alur utama yang diminta pada spesifikasi telah berjalan dengan benar. Pengujian dilakukan terhadap proses pembuatan vault, perlindungan local share, akses vault pada mode normal dan mode backup, perubahan isi vault, penyimpanan data pada server, kegagalan pemulihan, serta bonus kriptografi visual untuk recovery share. Setiap pengujian disusun berdasarkan skenario yang tercantum pada spesifikasi tugas, kemudian dianalisis untuk melihat apakah perilaku program sudah sesuai dengan kebutuhan sistem.

3.1 Uji Pembuatan Vault

Pengujian ini bertujuan untuk memastikan bahwa sistem dapat membuat vault baru dengan benar ketika pengguna memasukkan master password yang valid. Pada tahap ini, sistem harus membangkitkan master key acak, mengenkripsi vault kosong menggunakan AES-128-GCM, membagi master key menjadi tiga share dengan skema Shamir Secret Sharing (2, 3), serta menyimpan data awal ke komponen yang sesuai.

Skenario Pengujian

1. Pengguna membuat vault baru dengan master password yang valid.
2. Sistem membangkitkan master key secara acak.
3. Sistem membentuk vault kosong dan mengenkripsinya.
4. Sistem membagi master key menjadi local share, server share, dan recovery share.
5. Sistem menampilkan recovery share dalam format teks.
6. Server menyimpan server share, vault terenkripsi, nonce, dan metadata pengguna.

Hasil yang Diharapkan

1. Vault baru berhasil dibuat tanpa error.
2. Master key berhasil dibangkitkan.
3. Vault kosong berhasil dienkripsi.
4. Master key berhasil dibagi menjadi tiga share.
5. Recovery share tampil dalam format teks yang memuat koordinat dan nilai share.
6. Data yang tersimpan di server hanya berupa server share, vault terenkripsi, nonce, dan metadata.

Analisis Hasil

Berdasarkan pengujian yang dilakukan, sistem berhasil menjalankan alur pembuatan vault sesuai spesifikasi. Klien membangkitkan master key sepanjang 16 byte untuk AES-128, kemudian mengenkripsi vault kosong menggunakan AES-128-GCM. Setelah itu, master key dibagi menjadi tiga bagian menggunakan Shamir Secret Sharing dengan ambang batas (2, 3), yaitu local share, server share, dan recovery share. Recovery share ditampilkan kepada pengguna dalam format teks yang dapat disimpan sebagai cadangan.

Di sisi server, data awal yang diterima hanya berupa server share, vault terenkripsi, dan nonce vault, beserta identitas pengguna yang diperlukan untuk pengelolaan data. Dengan demikian, server tidak menyimpan master key secara utuh maupun isi vault dalam bentuk plaintext. Hasil ini menunjukkan bahwa proses pembuatan vault telah memenuhi rancangan sistem terdistribusi dan prinsip zero-knowledge yang ditetapkan pada spesifikasi.

```
Welcome to Credentials
.....
Let's get started.
Fill fields in any order.

1. Username*: nayaka
2. Master password*: ****
> 3. Save
4. Cancel

up/down move | enter select | q back | ctrl+c quit
```

```
Welcome to Save visual shares
.....
Let's get started.
Provide absolute output paths for the two recovery share images.
server: active

1. Visual share image 1 output path*: /mnt/c/Users/Ghana/Documents/sss-vault/img1.png
> 2. Visual share image 2 output path*: /mnt/c/Users/Ghana/Documents/sss-vault/img2.png
3. Save
4. Cancel

up/down move | enter select | q back | ctrl+c quit
```


Gambar 3.1-b Key yang dihasilkan

```

.....
1 function status() {
2 + vault created. Save this recovery share and visual shares now.
3 }
.....
eyJ4IjoyLCJ5IjoiQUs4QUdBQXlBQ29BUGdBbUFPMEFLZ0Q0QUVnQUFnQk1BR01BdHdCekFIND0ifQ==

share 1 saved to: /mnt/c/Users/Ghana/Documents/sss-vault/img1.png
share 2 saved to: /mnt/c/Users/Ghana/Documents/sss-vault/img2.png
temporary qr preview: /tmp/nayaka_recovery_preview_sj6qoyaz/recovery_qr.png
temporary overlay preview: /tmp/nayaka_recovery_preview_sj6qoyaz/overlay.png
> Press enter to continue: █

```

Gambar 3.1-c Recovery share

```

[~ wlntr from kaze][0.014s][RAM: 1/7GB][Sunday at 10:40:58 PM][main ≡]
[/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]
└─ sqlite3 data/server/vault.db
SQLite version 3.37.2 2022-01-06 13:25:41
Enter ".help" for usage hints.
sqlite> .headers on
sqlite> .mode column
sqlite> select * from users;
username  server_share                                vault_b
lob       vault_nonce  created_at      updated_at
-----
nayaka    eyJ4IjoyLCJ5IjoiQUVvQW13QTdBTzFBZXdCZEFIVUFRQUNWQUxzQVdRRDhBR29BVkFBSkFMcz0ifQ==  XH0JaY0
,000000v50W0c00i0 m00r+,R00  2026-06-07 15:35:12  2026-06-07 15:35:12
sqlite> █

```

Gambar 3.1-d Data yang disimpan server

3.2 Uji Penyimpanan dan Perlindungan *local share*

Pengujian ini bertujuan untuk memastikan bahwa local share tidak disimpan dalam bentuk asli pada sisi klien, melainkan dilindungi menggunakan kunci turunan dari master password. Selain itu, pengujian ini juga memverifikasi bahwa local share dapat didekripsi dengan master password yang benar, dan gagal dibuka ketika master password yang dimasukkan salah.

Skenario Pengujian

1. Periksa data local share yang tersimpan di sisi klien.
2. Verifikasi bahwa local share tersimpan dalam keadaan terenkripsi.
3. Masukkan master password yang benar.
4. Verifikasi bahwa local share berhasil didekripsi.
5. Masukkan master password yang salah.
6. Verifikasi bahwa dekripsi gagal atau akses ditolak.

Hasil yang Diharapkan

1. Local share tidak muncul dalam bentuk asli pada media penyimpanan klien.
2. Local share terenkripsi menggunakan kunci hasil KDF dari master password.
3. Master password yang benar dapat membuka local share.
4. Master password yang salah menyebabkan dekripsi gagal.

Analisis Hasil

Hasil pengujian menunjukkan bahwa local share tidak disimpan secara langsung dalam bentuk teks atau nilai share mentah, melainkan disimpan sebagai ciphertext bersama nonce dan salt KDF pada berkas klien. Pada implementasi ini, kunci untuk melindungi local share diturunkan dari master password menggunakan Argon2id. Dengan demikian, walaupun berkas klien diakses oleh pihak lain, local share tetap tidak dapat digunakan tanpa mengetahui master password yang benar.

Ketika pengguna memasukkan master password yang benar, sistem berhasil menurunkan kunci yang sesuai dan mendekripsi local share dengan benar. Sebaliknya, ketika pengguna memasukkan master password yang salah, proses dekripsi gagal sehingga akses tidak dapat dilanjutkan. Hal ini menunjukkan bahwa mekanisme perlindungan local share telah berjalan sesuai tujuan, yaitu menjaga agar salah satu share penting tetap aman walaupun data klien tersimpan secara lokal.

```

II4021-Tugas-4 > data > client > O nayaka.json > ...
1 {
2   "username": "nayaka",
3   "kdf": {
4     "name": "argon2id",
5     "iterations": 3,
6     "lanes": 4,
7     "memory_cost": 65536,
8     "length": 16
9   },
10  "salt": "ZiR0sQ8AfdFib1NDML6Ing==",
11  "encrypted_local_share": "Msxc2JEPsJtR37cJJIXhN32chaNCB8dRqD37uVxpzotKpMn8KND+1PuJCGNEQZSRdPoqsqBtUsAB7t43PC8ZDbNWSElHyTXjIWODl0LD1n18XmTRx3hdbvGps1yuskrV",
12  "local_share_nonce": "FbxZlWPx1C+w0Hdx",
13  "backup_vault": "5NgbkuFFDkiZAXSsIAHZet00/dsVymSE+x0F6yI/",
14  "backup_nonce": "I2DyU0NMGEs+vykL"
15 }

```

Gambar 3.2-a Data local share yang tersimpan di sisi klien

```

II4021-Tugas-4 > data > client > O nayaka.json > ...
1 {
2   "username": "nayaka",
3   "kdf": {
4     "name": "argon2id",
5     "iterations": 3,
6     "lanes": 4,
7     "memory_cost": 65536,
8     "length": 16
9   },
10  "salt": "ZiR0sQ8AfdFib1NDML6Ing==",
11  "encrypted_local_share": "Msxc2JEPsJtR37cJJIXhN32chaNCB8dRqD37uVxpzotKpMn8KND+1PuJCGNEQZSRdPoqsqBtUsAB7t43PC8ZDbNWSElHyTXjIWODl0LD1n18XmTRx3hdbvGps1yuskrV",
12  "local_share_nonce": "FbxZlWPx1C+w0Hdx",
13  "backup_vault": "5NgbkuFFDkiZAXSsIAHZet00/dsVymSE+x0F6yI/",
14  "backup_nonce": "I2DyU0NMGEs+vykL"
15 }

```

Gambar 3.2-b local share dienkripsi dari turunan password

```

Welcome to Open vault
.....
Let's get started.
Fill username and master password.
server: active

1. Username*: nayaka
2. Master password*: ****
> 3. Save
4. Cancel

up/down move | enter select | q back | ctrl+c quit

.....
1 function access() {
2 * normal mode: local share + server share
3 }

.....
> Press enter to continue: █

```

Gambar 3.2-c Local share berhasil didekripsi dari master password yang benar

```
Welcome to Open vault
.....
Let's get started.
Fill username and master password.
server: active

  1. Username*: nayaka
  2. Master password*: ***
> 3. Save
  4. Cancel

up/down move | enter select | q back | ctrl+c quit

.....
1 function error() {
2 - decryption failed
3 }
.....
> Press enter to continue: █
```

Gambar 3.2-c Local share gagal didekripsi dari master password yang salah

3.3 Uji Akses Normal

Pengujian ini bertujuan untuk memastikan bahwa vault dapat dibuka pada mode normal ketika server tersedia dan pengguna memasukkan master password yang benar. Pada mode ini, sistem harus menggunakan kombinasi local share dan server share untuk merekonstruksi master key, lalu menggunakan master key tersebut untuk mendekripsi vault.

Skenario Pengujian

1. Jalankan server dalam kondisi aktif.
2. Masukkan master password yang benar.
3. Sistem mengambil server share, vault terenkripsi, dan nonce dari server.
4. Sistem menggabungkan local share dan server share.
5. Sistem merekonstruksi master key dan mendekripsi vault.
6. Lakukan uji negatif dengan master password salah atau share tidak valid.

Hasil yang Diharapkan

1. Sistem menggunakan kombinasi local share dan server share.

2. Master key berhasil direkonstruksi.
3. Vault berhasil didekripsi dan isi password dapat ditampilkan.
4. Jika password salah atau share tidak valid, dekripsi gagal dan data tidak ditampilkan.

Analisis Hasil

Pada pengujian akses normal, sistem berhasil membuka vault ketika server dapat diakses dan pengguna memasukkan master password yang benar. Setelah local share berhasil didekripsi di sisi klien, sistem mengambil server share, vault terenkripsi, dan nonce vault dari server. Dua share tersebut kemudian digunakan untuk merekonstruksi master key, dan hasil rekonstruksi terbukti dapat dipakai untuk mendekripsi isi vault secara benar.

Selain pengujian positif, dilakukan juga pengujian negatif dengan menggunakan master password yang salah atau share yang tidak valid. Pada kondisi tersebut, proses rekonstruksi atau dekripsi tidak berhasil, sehingga isi vault tidak ditampilkan kepada pengguna. Hasil ini menunjukkan bahwa akses normal hanya dapat dilakukan apabila minimal dua share yang benar tersedia, sesuai dengan skema ambang batas (2, 3) yang digunakan.

```
[0 wlntr from 0 kaze][0 0.014s][0 RAM: 1/7GB][0 Sunday at 10:52:01 PM][0 }main ≡]
[/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]
└─Δ uv run python - <<'PY'
from getpass import getpass
from client.vault.manager import VaultManager
from client.crypto import Share, combine_shares, decrypt_bytes
from client.vault.codec import decode_entries

username = "nayaka"

with open("/dev/tty") as tty:
    import sys
    sys.stdin = tty
    master_password = getpass("Master password: ")

vm = VaultManager()

local_share = vm.unlock_local_share(username, master_password)
print("[1] local_share berhasil dibuka dari sisi klien")
print("    x =", local_share.x, "y_len =", len(local_share.y))

server_share_text = vm.server.load_server_share(username)
server_share = Share.from_text(server_share_text)
print("[2] server_share berhasil diambil dari server")
print("    x =", server_share.x, "y_len =", len(server_share.y))

vault_data = vm.server.load_vault_payload(username)
print("[3] vault terenkripsi dan nonce berhasil diambil dari server")
print("    vault_len =", len(vault_data["vault"]), "nonce_len =", len(vault_data["nonce"]))

master_key = combine_shares([local_share, server_share])
print("[4] master key berhasil direkonstruksi dari dua share valid")
print("    master_key_len =", len(master_key))
print("    master_key_hex =", master_key.hex())

plaintext = decrypt_bytes(master_key, vault_data["vault"], vault_data["nonce"])
entries = decode_entries(plaintext)
print("[5] vault berhasil didekripsi")
print("    jumlah entri =", len(entries))

for i, entry in enumerate(entries, 1):
    print(f"    entry {i}: service={entry.service}, username={entry.username}, notes={entry.notes}")
```

```
for i, entry in enumerate(entries, 1):
    print(f"    entry {i}: service={entry.service}, username={entry.username}, notes={entry.notes}")
)
PY
Master password:
[1] local_share berhasil dibuka dari sisi klien
    x = 1 y_len = 32
[2] server_share berhasil diambil dari server
    x = 2 y_len = 32
[3] vault terenkripsi dan nonce berhasil diambil dari server
    vault_len = 30 nonce_len = 12
[4] master key berhasil direkonstruksi dari dua share valid
    master_key_len = 16
    master_key_hex = 81a04dbbf5cb865bd0a0065a788f3634
[5] vault berhasil didekripsi
    jumlah entri = 0
```

Gambar 3.3-a Penggunaan local share + server share pada sistem

```
for i, entry in enumerate(entries, 1):
    print(f"    entry {i}: service={entry.service}, username={entry.username}, notes={entry.notes}")
)
PY
Master password:
[1] local_share berhasil dibuka dari sisi klien
    x = 1 y_len = 32
[2] server_share berhasil diambil dari server
    x = 2 y_len = 32
[3] vault terenkripsi dan nonce berhasil diambil dari server
    vault_len = 30 nonce_len = 12
[4] master key berhasil direkonstruksi dari dua share valid
    master_key_len = 16
    master_key_hex = 81a04dbbf5cb865bd0a0065a788f3634
[5] vault berhasil didekripsi
    jumlah entri = 0
```

Gambar 3.3-b Master key berhasil direkonstruksi

```
for i, entry in enumerate(entries, 1):
    print(f"    entry {i}: service={entry.service}, username={entry.username}, notes={entry.notes}")
)
PY
Master password:
[1] local_share berhasil dibuka dari sisi klien
    x = 1 y_len = 32
[2] server_share berhasil diambil dari server
    x = 2 y_len = 32
[3] vault terenkripsi dan nonce berhasil diambil dari server
    vault_len = 30 nonce_len = 12
[4] master key berhasil direkonstruksi dari dua share valid
    master_key_len = 16
    master_key_hex = 81a04dbbf5cb865bd0a0065a788f3634
[5] vault berhasil didekripsi
    jumlah entri = 0
```

Gambar 3.3-c Vault berhasil didekripsi

```

master_key = combine_shares([local_share, server_share])
print("[4] master key berhasil direkonstruksi dari dua share valid")
print("    master_key_len =", len(master_key))
print("    master_key_hex =", master_key.hex())

plaintext = decrypt_bytes(master_key, vault_data["vault"], vault_data["nonce"])
entries = decode_entries(plaintext)
print("[5] vault berhasil didekripsi")
print("    jumlah entri =", len(entries))

for i, entry in enumerate(entries, 1):
    print(f"    entry {i}: service={entry.service}, username={entry.username}, notes={entry.notes}"
    )
PY
Master password:
Traceback (most recent call last):
  File "/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4/src/client/crypto/aes_gcm.py", line 2
1, in decrypt_bytes
    return AESGCM(key).decrypt(nonce, ciphertext, None)
    ~~~~~^~~~~~
cryptography.exceptions.InvalidTag

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "<stdin>", line 15, in <module>
  File "/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4/src/client/vault/manager.py", line 11
3, in unlock_local_share
    plaintext = decrypt_bytes(
        password_key,
        client_data["encrypted_local_share"],
        client_data["local_share_nonce"],
    )
  File "/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4/src/client/crypto/aes_gcm.py", line 2
3, in decrypt_bytes
    raise ValueError("decryption failed") from exc
ValueError: decryption failed
[0 wlntr from 0 kaze][0 2.845s][0 RAM: 1/7GB][0 Sunday at 10:55:53 PM][0 ʘmain ≡][0 Error, check y
our command]
[0 /mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]
[0 Δ

```

Gambar 3.4-c Vault gagal didekripsi

3.4 Uji Penambahan Data Password

Pengujian ini dilakukan untuk memastikan bahwa pengguna dapat menambahkan data password baru ke dalam vault pada mode normal, baik dengan memasukkan password secara manual maupun dengan menggunakan pembangkitan otomatis berbasis CSPRNG. Setelah perubahan dilakukan, sistem harus mengenkripsi ulang vault dengan nonce baru dan memperbarui data pada server serta backup vault lokal.

Skenario Pengujian

1. Buka vault pada mode normal.
2. Tambahkan entri password dengan input manual.

3. Tambahkan entri password dengan pembangkitan otomatis.
4. Verifikasi bahwa entri baru muncul pada isi vault.
5. Verifikasi bahwa vault dienkrpsi ulang dengan nonce baru.
6. Verifikasi bahwa server dan backup vault lokal ikut diperbarui.

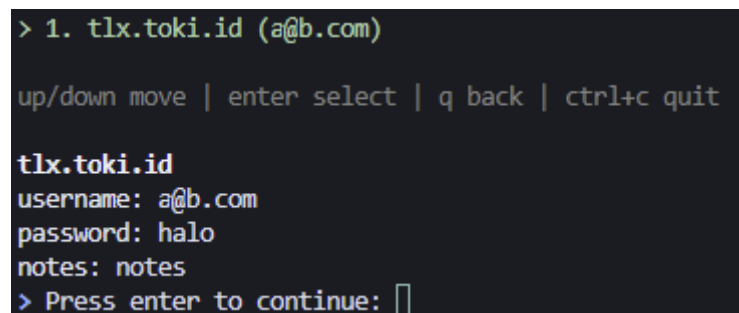
Hasil yang Diharapkan

1. Entri password manual berhasil ditambahkan.
2. Password otomatis berhasil dibangkitkan sesuai panjang yang diminta.
3. Isi vault bertambah sesuai data baru.
4. Vault dienkrpsi ulang dengan nonce baru.
5. Perubahan tersimpan ke server dan backup vault lokal.

Analisis Hasil

Hasil pengujian menunjukkan bahwa sistem berhasil menambahkan data password baru ke dalam vault pada mode normal. Pada skenario pertama, pengguna mengisi nama layanan, username, password, dan catatan secara manual, lalu entri tersebut muncul pada daftar isi vault setelah disimpan. Pada skenario kedua, sistem juga berhasil membangkitkan password otomatis menggunakan CSPRNG sesuai panjang yang diminta pengguna, kemudian menyimpannya sebagai bagian dari entri baru.

Setelah penambahan data dilakukan, sistem tidak hanya memperbarui isi vault di memori klien, tetapi juga mengenkripsi ulang seluruh vault sebagai satu kesatuan menggunakan nonce baru. Hasil enkripsi baru tersebut kemudian dikirim kembali ke server, dan salinan backup vault lokal juga diperbarui. Hal ini menunjukkan bahwa mekanisme sinkronisasi data pada mode normal telah berjalan sesuai spesifikasi.



```
> 1. tlx.toki.id (a@b.com)

up/down move | enter select | q back | ctrl+c quit

tlx.toki.id
username: a@b.com
password: halo
notes: notes
> Press enter to continue: 
```

Gambar 3.4-a Penambahan data password dengan input manual

```
1. tlx.toki.id (a@b.com)
> 2. codechef.com (a@b.com)

up/down move | enter select | q back | ctrl+c quit

codechef.com
username: a@b.com
password: *gdU0?rc@e;=@p#sZ;!?hIo$0lm++Ih&m)Z087CC8YoL8AV{bGNa9DPSU{n}6C.C3n041}!F-D!{5SXduJLVUhp0n;o0Crsp.8D;UZvhiN4[yu0!8D:_y2T=c$c}md5
notes: bandung
> Press enter to continue: █
```

Gambar 3.4-b Penambahan data password dengan pembangkitan otomatis

```
{
  "username": "apapun",
  "kdf": {
    "name": "argon2id",
    "iterations": 3,
    "lanes": 4,
    "memory_cost": 65536,
    "length": 16
  },
  "salt": "IjTZBQiWT5s3n3ef7tykag==",
  "encrypted_local_share": "kvx5CsVCnhL8VP4tt/oS3U0adNKygGfWctQZJ27TQEDg678o1Jdcf",
  "local_share_nonce": "bQ00q/OCXWi85D1J",
  "backup_vault": "GC3m45oecVnXv9YcOJI20Gy0qA7sbx2I5B5C+A/n",
  "backup_nonce": "/iR1bdPDQ2GFM3Zk"
}
```

Gambar 3.4-c Data backup lokal sebelum penambahan entri baru

```
{
  "username": "apapun",
  "kdf": {
    "name": "argon2id",
    "iterations": 3,
    "lanes": 4,
    "memory_cost": 65536,
    "length": 16
  },
  "salt": "IjTZBQiWT5s3n3ef7tykag==",
  "encrypted_local_share": "kvx5CsVCnhL8VP4tt/oS3U0adNKygGfWctQZJ27TQEDg678o1JdcBMDJFg3sIRj845905jMoCz0s6tBXg+SjaJzGhoiPFH/+HYv3rIcyTwni9Au12q1hXk0u1Rzb11Au",
  "local_share_nonce": "bQ00q/OCXWi85D1J",
  "backup_vault": "2n814I91haCkUQND3pQD1mmHr+xdgS51u+1TugBwmGbRcLrD+hVBg49IQPw1HuKvoKv60PQPjxKRPdhgK+88xBV7YrgqdZT8R164e1720Tr+yZn2L6KgmVAX89KkZvTg+bVIG9/jXJBG",
  "backup_nonce": "AEVe6ZxS2a60q+UJ"
}
```

Gambar 3.4-d Data backup lokal setelah penambahan entri baru

3.5 Uji Pengubahan dan Penghapusan Data Password

Pengujian ini bertujuan untuk memastikan bahwa data password yang sudah tersimpan dapat diubah maupun dihapus pada mode normal. Selain itu, pengujian ini juga

memverifikasi bahwa pembaruan hanya diperbolehkan pada mode normal dan tidak berlaku pada mode backup.

Skenario Pengujian

1. Buka vault pada mode normal.
2. Ubah salah satu entri yang sudah ada.
3. Simpan perubahan dan buka ulang vault.
4. Hapus salah satu entri yang tersimpan.
5. Buka ulang vault dan verifikasi bahwa entri tersebut tidak lagi muncul.
6. Pastikan pembaruan tidak dapat dilakukan pada mode backup.

Hasil yang Diharapkan

1. Data password dapat diubah pada mode normal.
2. Data hasil perubahan tetap tersimpan setelah vault dibuka ulang.
3. Data password dapat dihapus pada mode normal.
4. Data yang dihapus tidak lagi muncul.
5. Operasi ubah dan hapus ditolak pada mode backup.

Analisis Hasil

Berdasarkan pengujian, sistem berhasil melakukan pengubahan dan penghapusan data password selama vault dibuka dalam mode normal. Ketika salah satu entri diubah, perubahan tersebut tersimpan dengan benar dan tetap muncul setelah vault ditutup lalu dibuka kembali. Hal yang sama juga berlaku untuk penghapusan data, di mana entri yang dihapus tidak lagi muncul ketika isi vault dimuat ulang.

Setelah operasi ubah atau hapus dilakukan, sistem mengenkripsi ulang seluruh isi vault dan memperbarui data terenkripsi pada server serta backup lokal. Sebaliknya, ketika pengguna mencoba melakukan operasi modifikasi pada mode backup, sistem menolak permintaan tersebut karena mode backup bersifat read-only. Hasil ini sesuai dengan spesifikasi yang menyatakan bahwa perubahan data hanya boleh dilakukan pada mode normal untuk mencegah konflik versi data.

```
> 1. tlx.toki.id (someone@gmail.com)

up/down move | enter select | q back | ctrl+c quit

tlx.toki.id
username: someone@gmail.com
password: Pass
notes: notes
> Press enter to continue: █
```

```
> 1. tlx.toki.id (something@gmail.com)

up/down move | enter select | q back | ctrl+c quit

tlx.toki.id
username: something@gmail.com
password: Pass
notes: notes
> Press enter to continue: █
```

Gambar 3.5-a Pengubahan salah satu entri password

```
1. tlx.toki.id (something@gmail.com)
> 2. atcoder.jp (someone@gmail.com)

up/down move | enter select | q back | ctrl+c quit

atcoder.jp
username: someone@gmail.com
password: A;P1b.ithwD;#k2+
notes: nothing
> Press enter to continue: █
```

```
> 1. atcoder.jp (someone@gmail.com)

up/down move | enter select | q back | ctrl+c quit

atcoder.jp
username: someone@gmail.com
password: A;P1b.ithwD;#k2+
notes: nothing
> Press enter to continue: █
```

Gambar 3.5-b Penghapusan salah satu entri password

```
server: inactive | mode: backup | create vault disabled

> 1. Recovery share*: [empty]
  2. Save
  3. Cancel

up/down move | enter select | q back | ctrl+c quit
> Recovery share: eyJ4IjozLCJ5IjoiaQVA4QXJnRDZBT01B0HdEZUFQb0FoQUFMQUxRQXRRQVZBTWdBdHdDeUFBZz0ifQ==
```

```
Let's get started.
Backup mode is read-only.
server: inactive | mode: backup | create vault disabled

> 1. View entry
  2. Back

up/down move | enter select | q back | ctrl+c quit
```

Gambar 3.5-c Penolakan perubahan data pada mode backup

3.6 Uji Penyimpanan Vault di Server

Pengujian ini dilakukan untuk memastikan bahwa server hanya menyimpan data yang memang diperbolehkan oleh spesifikasi, yaitu server share, vault terenkripsi, nonce, dan metadata pengguna. Selain itu, pengujian ini juga memverifikasi bahwa vault disimpan di SQLite sebagai BLOB dan bahwa server tidak menyimpan isi vault dalam bentuk plaintext.

Skenario Pengujian

1. Buat atau perbarui vault melalui klien.
2. Periksa isi tabel SQLite pada server.
3. Verifikasi tipe dan bentuk data yang disimpan.
4. Pastikan tidak ada nama layanan, username, password, atau catatan dalam bentuk plaintext.
5. Pastikan server tidak menerima master key, local share, recovery share, atau isi vault asli.

Hasil yang Diharapkan

1. Vault terenkripsi tersimpan di SQLite sebagai BLOB.
2. Nonce vault dan server share tersimpan sesuai kebutuhan akses normal.
3. Tidak ada atribut isi vault yang tersimpan sebagai baris plaintext terpisah.
4. Server tidak menyimpan master key, local share, atau recovery share.

Analisis Hasil

Hasil pengujian menunjukkan bahwa server menyimpan data vault sesuai dengan prinsip zero-knowledge yang dirancang dalam sistem. Pada basis data SQLite, informasi yang tersimpan hanya mencakup identitas pengguna, server share, vault_blob, dan vault_nonce. Nilai vault_blob tersimpan dalam bentuk BLOB sehingga isi vault tidak dapat dibaca langsung tanpa proses dekripsi di sisi klien.

Selain itu, tidak ditemukan penyimpanan atribut seperti nama layanan, username, password, atau catatan sebagai data terpisah dalam bentuk plaintext. Server juga tidak menerima master key, local share, recovery share, maupun kunci turunan dari master password. Dengan demikian, pengujian ini menunjukkan bahwa penyimpanan data di server telah sesuai dengan batasan keamanan yang ditentukan pada spesifikasi tugas.

```

1 {
2   "username": "b",
3   "kdf": {
4     "name": "argon2id",
5     "iterations": 3,
6     "lanes": 4,
7     "memory_cost": 65536,
8     "length": 16
9   },
10  "salt": "TAKPaXbpKOPDa1VGC+rPQ--",
11  "encrypted_local_share": "j3k/cp4D09j3q30clpgs+svvMptaRET3LU8mHZfuqzjqvV54JnK3+ZopZuCcn7gHHDQ1tJe5KDoexxTAQvL105aYzTpINgLRrMAU1pd62eA4bPwOXHvaJ3DYmX1o",
12  "local_share_nonce": "qXewgBgtqscbDV",
13  "backup_vault": "ce0IO+/Al33uzE8XKX6/Ghe9ZhJLpRLH6KMYVPqXpFtgvI43Hua6G+KqDy62++L6M9OUZAVc9IMfgdASKu3FXT0V56BRq5ShY454TUzdk8LFPRep85yw/b90mDCNKV5o5scr/1rXN+hd1Ij4KonpIpxLmh7prtI-",
14  "backup_nonce": "RPA9WN9zY3UVhq/f"
15 }

```

Gambar 3.6-a Struktur data yang tersimpan pada SQLite server

```

(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "PRAGMA table_info(users);
>>
+
0|id|INTEGER|0|1
1|username|TEXT|1||0
2|server_share|TEXT|1||0
3|vault_blob|TEXT|1||0
4|vault_nonce|TEXT|1||0
5|created_at|TEXT|0||0
6|updated_at|TEXT|0||0
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT username, typeof(vault_blob) AS blob_type, typeof(vault_nonce) AS nonce_type, length(vault_blob) AS
blob_len, length(vault_nonce) AS nonce_len, length(server_share) AS share_len FROM users;"
smoketest_user|blob|2|2|5
ben|blob|blob|30|12|80
Benedict|blob|blob|241|12|80
Presley|blob|blob|38|12|80
p|blob|blob|138|12|80
b|blob|blob|122|12|80
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT cast(vault_blob AS text) FROM users WHERE username='b';"
95h v 42944m
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT COUNT(*) FROM users WHERE cast(vault_blob AS text) LIKE 'XtestPlaintext36%';"
0
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT COUNT(*) FROM users WHERE cast(vault_blob AS text) LIKE 'X@th.ac.id%';"
0
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT COUNT(*) FROM users WHERE cast(vault_blob AS text) LIKE 'XPass36%';"
0
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT COUNT(*) FROM users WHERE cast(vault_blob AS text) LIKE 'X36%';"
0
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db ".tables"
users
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT * FROM users WHERE username='b';"
6|b|ey4j4joytC51joiQus4QZuqnl85j8BdmCREf7Z0EzQuXQJQJ38DIBSFV8Tgd8QUFIVt01fQ==|95h v 42944m|4m Z|4444e| 2026-06-07 16:56:21

```

Gambar 3.6-b Vault terenkripsi yang tersimpan sebagai BLOB

```
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT cast(vault_blob AS text) FROM users WHERE username='b';"
b3v v #2L9#W
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT COUNT(*) FROM users WHERE cast(vault_blob AS text) LIKE '%TestPlaintext36%';"
0
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT COUNT(*) FROM users WHERE cast(vault_blob AS text) LIKE '%@itb.ac.id%';"
0
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT COUNT(*) FROM users WHERE cast(vault_blob AS text) LIKE '%Pass36%';"
0
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT COUNT(*) FROM users WHERE cast(vault_blob AS text) LIKE '%36%';"
0
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db ".tables"
users
(II4021-Tugas-4) PS C:\Users\Benedict Presley\Semester6\Kriptografi\II4021-Tugas-4> sqlite3 data/server/vault.db "SELECT * FROM users WHERE username='b';"
users
```

Gambar 3.6-c Verifikasi bahwa server tidak menyimpan isi vault dalam plaintext

3.7 Uji Mode Backup

Pengujian ini bertujuan untuk memastikan bahwa sistem tetap dapat membuka vault dalam kondisi darurat ketika server tidak dapat diakses. Pada mode backup, sistem harus menggunakan kombinasi local share dan recovery share, serta menggunakan backup vault lokal terenkripsi sebagai sumber data. Mode ini juga harus membatasi pengguna agar hanya dapat melihat isi vault tanpa melakukan perubahan.

Skenario Pengujian

1. Simulasikan server dalam kondisi tidak dapat diakses.
2. Masukkan master password untuk membuka local share.
3. Masukkan recovery share.
4. Sistem merekonstruksi master key dari local share dan recovery share.
5. Sistem membuka backup vault lokal.
6. Coba lakukan operasi tambah, ubah, atau hapus data.

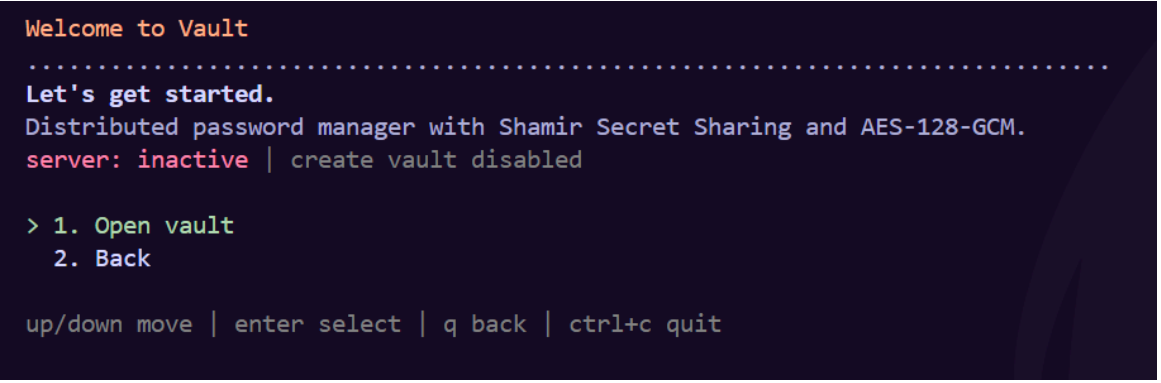
Hasil yang Diharapkan

1. Vault tetap dapat dibuka tanpa koneksi ke server.
2. Sistem menggunakan kombinasi local share dan recovery share.
3. Sumber data berasal dari backup vault lokal terenkripsi.
4. Data password dapat dilihat.
5. Operasi tambah, ubah, dan hapus ditolak.

Analisis Hasil

Pada pengujian ini, sistem berhasil menjalankan mode backup ketika server dibuat tidak tersedia. Setelah pengguna memasukkan master password yang benar dan recovery share yang valid, sistem menggunakan local share serta recovery share untuk merekonstruksi master key. Kunci tersebut kemudian dipakai untuk mendekripsi backup vault lokal yang sebelumnya telah disimpan dalam keadaan terenkripsi.

Isi vault dapat ditampilkan kepada pengguna, sehingga mode backup tetap memenuhi kebutuhan akses darurat. Namun, ketika pengguna mencoba melakukan penambahan, pengubahan, atau penghapusan data, sistem menolak operasi tersebut karena mode backup bersifat read-only. Hasil pengujian ini menunjukkan bahwa mekanisme pemulihan darurat telah berjalan sesuai spesifikasi dan tetap menjaga konsistensi data dengan server.

A screenshot of a terminal window with a dark background and light-colored text. The text shows the Vault CLI welcome message and a prompt to get started. It indicates that the server is inactive and that the create vault feature is disabled. A menu is shown with two options: '1. Open vault' and '2. Back'. At the bottom, there are instructions for navigation: 'up/down move', 'enter select', 'q back', and 'ctrl+c quit'.

```
Welcome to Vault
.....
Let's get started.
Distributed password manager with Shamir Secret Sharing and AES-128-GCM.
server: inactive | create vault disabled

> 1. Open vault
   2. Back

up/down move | enter select | q back | ctrl+c quit
```

Gambar 3.7-a Simulasi kondisi server tidak dapat diakses


```
II4021-Tugas-4 > recov
1 eyJ4IjozLCJ5IjoIQUs4QUdBQXlBQ29BUGdBbUFPMEFLZ0Q0QUVnQUFnQk1BR01BdHdCekFIND0ifQ==

OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
v TERMINAL  wsl ⚠ + v
└─[mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]
└─Δ uv run python - <<'PY'
from getpass import getpass
from pathlib import Path
import json
import sys

from client.vault.manager import VaultManager
from client.crypto import Share
from shared.models import VaultEntry

username = "nayaka"

with open("/dev/tty") as tty:
    sys.stdin = tty
    master_password = getpass("Master password: ")

recovery_share_text = Path("recov").read_text(encoding="utf-8").strip()

vm = VaultManager()

print("[1] Membuka local share dari penyimpanan klien")
local_share = vm.unlock_local_share(username, master_password)
print("    local_share.x =", local_share.x)
print("    local_share.y_len =", len(local_share.y))

print("[2] Membaca recovery share dari file cadangan pengguna")
recovery_share = Share.from_text(recovery_share_text)
print("    recovery_share.x =", recovery_share.x)
print("    recovery_share.y_len =", len(recovery_share.y))

print("[3] Memeriksa bahwa sumber vault backup ada di penyimpanan lokal klien")
client_raw = json.loads(Path(f"data/client/{username}.json").read_text(encoding="utf-8"))
print("    backup_vault exists =", "backup_vault" in client_raw)
print("    backup_nonce exists =", "backup_nonce" in client_raw)
```

```

print("[3] Memeriksa bahwa sumber vault backup ada di penyimpanan lokal klien")
client_raw = json.loads(Path(f"data/client/{username}.json").read_text(encoding="utf-8"))
print("    backup_vault exists =", "backup_vault" in client_raw)
print("    backup_nonce exists =", "backup_nonce" in client_raw)
PY print("    delete_entry ditolak:", exc)rena tidak ada entri")ername="u2", password="p2", notes=
Master password:
[1] Membuka local share dari penyimpanan klien
    local_share.x = 1
    local_share.y_len = 32
[2] Membaca recovery share dari file cadangan pengguna
    recovery_share.x = 3
    recovery_share.y_len = 32
[3] Memeriksa bahwa sumber vault backup ada di penyimpanan lokal klien
    backup_vault exists = True
    backup_nonce exists = True
    encrypted_local_share exists = True
[4] Membuka vault dengan kombinasi local share + recovery share
    mode = backup
    readonly = True
    jumlah entri = 1
[5] Menampilkan isi password dari backup vault
    entry 1: service=github, username=naykaa, password=ghanaganteng, notes=abcdefgh
[6] Menguji bahwa mode backup tidak mengizinkan operasi tulis
    add_entry ditolak: backup mode is read-only
    update_entry ditolak: backup mode is read-only
    delete_entry ditolak: backup mode is read-only
[0] wlntr from kaze[0 4.702s][0 RAM: 1/7GB][0 Sunday at 11:03:25 PM][0 main ≡ 0 ?1]
[/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]
└─┐

```

Gambar 3.7-b Kombinasi local share dan recovery share pada sistem

```

"backup_vault": "Ky+buFXMVIXI680r7eIm7Sb75w/2Q5zOTKa47MNXJD3tGgkwz7BxxgNMGpeZVwC/unz7A3XoHmTjD0p/awIUDSGNzgff+Axe7
"backup_nonce": "TXplIs/Bj/yd7+53"
}

```

```

print("[3] Memeriksa bahwa sumber vault backup ada di penyimpanan lokal klien")
client_raw = json.loads(Path(f"data/client/{username}.json").read_text(encoding="utf-8"))
print("    backup_vault exists =", "backup_vault" in client_raw)
print("    backup_nonce exists =", "backup_nonce" in client_raw)
PY print("    delete_entry ditolak:", exc)rena tidak ada entri")ername="u2", password="p2", notes=
Master password:
[1] Membuka local share dari penyimpanan klien
    local_share.x = 1
    local_share.y_len = 32
[2] Membaca recovery share dari file cadangan pengguna
    recovery_share.x = 3
    recovery_share.y_len = 32
[3] Memeriksa bahwa sumber vault backup ada di penyimpanan lokal klien
    backup_vault exists = True
    backup_nonce exists = True
    encrypted_local_share exists = True
[4] Membuka vault dengan kombinasi local share + recovery share
    mode = backup
    readonly = True
    jumlah entri = 1
[5] Menampilkan isi password dari backup vault
    entry 1: service=github, username=naykaa, password=ghanaganteng, notes=abcdefgh
[6] Menguji bahwa mode backup tidak mengizinkan operasi tulis
    add_entry ditolak: backup mode is read-only
    update_entry ditolak: backup mode is read-only
    delete_entry ditolak: backup mode is read-only
[?] wIntr from kaze][?] 4.702s][?] RAM: 1/7GB][?] Sunday at 11:03:25 PM][?] main ≡ ?1]
[/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]
└─┐

```

Gambar 3.7-c Backup vault terenkripsi

```

Welcome to Open vault
.....
Let's get started.
Enter recovery share text to continue in backup mode.
server: inactive | mode: backup | create vault disabled

1. Recovery share*: eyJ4IjozLCJ5IjoIQUs4QUdBQX1BQ29BUGdBbUFPMEFLZ0Q0QUVnQUFnQk1BR01BdHdCekFIND0if
> 2. Save
3. Cancel

up/down move | enter select | q back | ctrl+c quit

.....
1 function access() {
2 * backup mode: local share + recovery share
3 }
.....
> Press enter to continue:

```

```
Welcome to View entry
.....
Let's get started.
Use arrow keys and enter.

> 1. github (naykaa)

up/down move | enter select | q back | ctrl+c quit

github
username: naykaa
password: ghanaganteng
notes: abcdefgh
> Press enter to continue: █
```

Gambar 3.7-d Password dapat dilihat pada backup mode

```
Welcome to nayaka vault (backup)
.....
Let's get started.
Backup mode is read-only.
server: inactive | mode: backup | create vault disabled

> 1. View entry
   2. Back

up/down move | enter select | q back | ctrl+c quit
```

Gambar 3.7-e Hanya bisa melihat entry pada backup mode

3.8 Uji Kegagalan Pemulihan

Pengujian ini dilakukan untuk memastikan bahwa sistem menolak proses pemulihan apabila recovery share yang dimasukkan salah atau apabila backup vault yang digunakan tidak sesuai atau telah dimodifikasi. Dalam kondisi tersebut, proses rekonstruksi master key atau dekripsi AES-128-GCM harus gagal, dan sistem tidak boleh menampilkan isi vault kepada pengguna.

Skenario Pengujian

1. Masukkan recovery share yang salah.
2. Coba buka vault pada mode backup.
3. Gunakan backup vault yang tidak sesuai atau telah dimodifikasi.
4. Coba lakukan dekripsi kembali.
5. Verifikasi bahwa data password tidak ditampilkan.

Hasil yang Diharapkan

1. Recovery share yang salah menyebabkan rekonstruksi atau dekripsi gagal.
2. Backup vault yang dimodifikasi menyebabkan validasi autentikasi AES-GCM gagal.
3. Sistem tidak menampilkan data password jika pemulihan gagal.

Analisis Hasil

Hasil pengujian menunjukkan bahwa sistem mampu mendeteksi kegagalan pemulihan dengan benar. Ketika pengguna memasukkan recovery share yang salah, master key yang direkonstruksi menjadi tidak sesuai, sehingga proses dekripsi backup vault gagal. Akibatnya, sistem tidak menampilkan isi vault dan akses tidak diberikan kepada pengguna.

Pengujian kedua dilakukan dengan menggunakan backup vault yang tidak sesuai atau telah dimodifikasi. Karena vault dilindungi menggunakan AES-128-GCM, perubahan sekecil apa pun pada ciphertext atau nonce menyebabkan validasi autentikasi gagal saat dekripsi. Sistem merespons kondisi tersebut dengan menolak pembukaan vault dan tidak menampilkan data apa pun. Hal ini menunjukkan bahwa mekanisme autentikasi bawaan pada AES-GCM telah berfungsi untuk menjaga integritas data vault.

```

└─Δ uv run python - <<'PY'
from getpass import getpass
from pathlib import Path
import sys

from client.vault.manager import VaultManager

username = "nayaka"

with open("/dev/tty") as tty:
    sys.stdin = tty
    master_password = getpass("Master password: ")

wrong_recovery = Path("recov").read_text(encoding="utf-8").strip()

vm = VaultManager()

print("[1] Local share dibuka dengan master password yang benar")
local_share = vm.unlock_local_share(username, master_password)
print("    local_share.x =", local_share.x)

print("[2] Menggunakan recovery share yang salah dari file recov")

try:
    vault = vm.open_backup_with_local_share(username, local_share, wrong_recovery)
    print("ERROR: seharusnya vault gagal dibuka")
    print("jumlah entri =", len(vault.entries))
except Exception as exc:
    print("[3] Rekonstruksi master key atau dekripsi backup vault gagal")
    print("    error =", exc)
    print("[4] Isi password tidak ditampilkan karena vault tidak berhasil dibuka")
PY
Master password:
[1] Local share dibuka dengan master password yang benar
    local_share.x = 1
[2] Menggunakan recovery share yang salah dari file recov
[3] Rekonstruksi master key atau dekripsi backup vault gagal
    error = invalid share value
[4] Isi password tidak ditampilkan karena vault tidak berhasil dibuka
[0 wlntr from 0 kaze][0 4.629s][0 RAM: 1/7GB][0 Sunday at 11:09:54 PM][0 ?main ≡ 0 ?2]
└─[/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]

```

Gambar 3.8-a Kegagalan pemulihan akibat recovery share tidak valid

```
[wintr from kaze][0.012s][RAM: 1/7GB][Sunday at 11:13:18 PM][main == ?3]
[/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]
uv run python - <<'PY'
from getpass import getpass
from pathlib import Path
import sys

from client.vault.manager import VaultManager

username = "nayaka"

with open("/dev/tty") as tty:
    sys.stdin = tty
    master_password = getpass("Master password: ")

recovery_share_text = Path("recov").read_text(encoding="utf-8").strip()

vm = VaultManager()

print("[1] Local share dibuka dengan master password yang benar")
PY print("[5] Isi password tidak ditampilkan karena proses dekripsi gagal")(kasi")text)
Master password:
[1] Local share dibuka dengan master password yang benar
    local_share.x = 1
[2] Recovery share yang valid dibaca dari file recov
    recovery_share length = 80
[3] Sistem mencoba membuka backup vault lokal terenkripsi
[4] Dekripsi AES-128-GCM gagal karena backup vault telah dimodifikasi
    error = invalid share value
[5] Isi password tidak ditampilkan karena proses dekripsi gagal
[wintr from kaze][4.68s][RAM: 1/7GB][Sunday at 11:13:24 PM][main == ?3]
[/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]
uv
```

Gambar 3.8-b Kegagalan dekripsi akibat backup vault tidak sesuai

```

[~ wlntr from ~ kaze][~ 0.012s][~ RAM: 1/7GB][~ Sunday at 11:13:18 PM][~ ~main ~ ~ ?3]
[/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]
└─Δ uv run python - <<'PY'
from getpass import getpass
from pathlib import Path
import sys

from client.vault.manager import VaultManager

username = "nayaka"

with open("/dev/tty") as tty:
    sys.stdin = tty
    master_password = getpass("Master password: ")

recovery_share_text = Path("recov").read_text(encoding="utf-8").strip()

vm = VaultManager()

print("[1] Local share dibuka dengan master password yang benar")
PY print("[5] Isi password tidak ditampilkan karena proses dekripsi gagal")kasi")text)
Master password:
[1] Local share dibuka dengan master password yang benar
    local_share.x = 1
[2] Recovery share yang valid dibaca dari file recov
    recovery_share length = 80
[3] Sistem mencoba membuka backup vault lokal terenkripsi
[4] Dekripsi AES-128-GCM gagal karena backup vault telah dimodifikasi
    error = invalid share value
[5] Isi password tidak ditampilkan karena proses dekripsi gagal
[~ wlntr from ~ kaze][~ 4.68s][~ RAM: 1/7GB][~ Sunday at 11:13:24 PM][~ ~main ~ ~ ?3]
[/mnt/c/Users/Ghana/Documents/sss-vault/II4021-Tugas-4]
└─Δ

```

Gambar 3.8-c Sistem tidak menampilkan isi vault ketika pemulihan gagal

3.9 Uji Kriptografi Visual

Pengujian bonus ini bertujuan untuk memastikan bahwa recovery share dapat diubah menjadi QR code, dibagi menjadi dua gambar share dengan skema visual (2, 2), lalu digabungkan kembali sehingga QR code hasil rekonstruksi masih dapat dipindai dan menghasilkan recovery share yang sama. Pengujian ini dilakukan untuk memverifikasi bahwa fitur bonus kriptografi visual telah bekerja sesuai rancangan.

Skenario Pengujian

1. Bangkitkan QR code dari recovery share.
2. Bagi QR code menjadi dua gambar share.
3. Tampilkan kedua gambar share.
4. Gabungkan kedua gambar share.
5. Pindai QR code hasil penggabungan.

6. Verifikasi bahwa hasil pemindaian sama dengan recovery share awal.

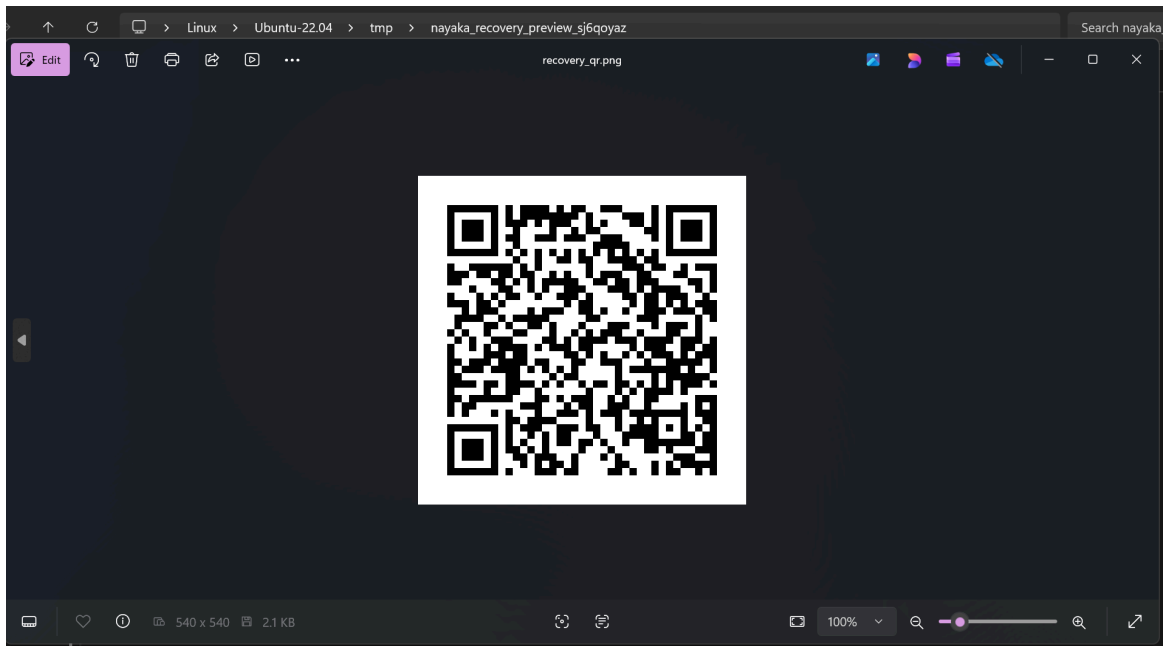
Hasil yang Diharapkan

1. QR code awal berhasil dibuat dari recovery share.
2. Dua gambar share berhasil dibentuk.
3. Hasil penggabungan kedua gambar menghasilkan QR code yang dapat dipindai.
4. Nilai recovery share hasil pemindaian sama dengan nilai awal.

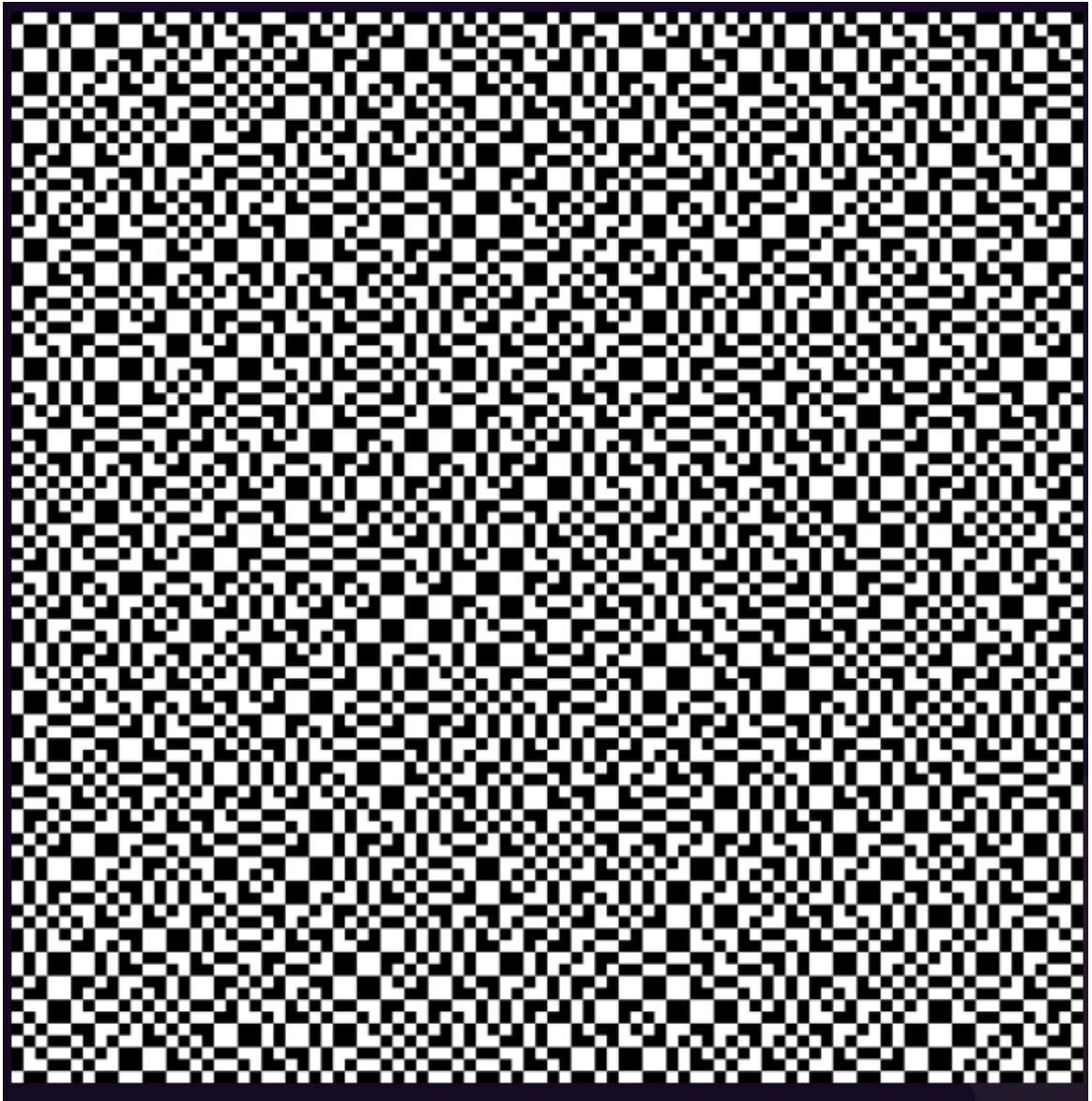
Analisis Hasil

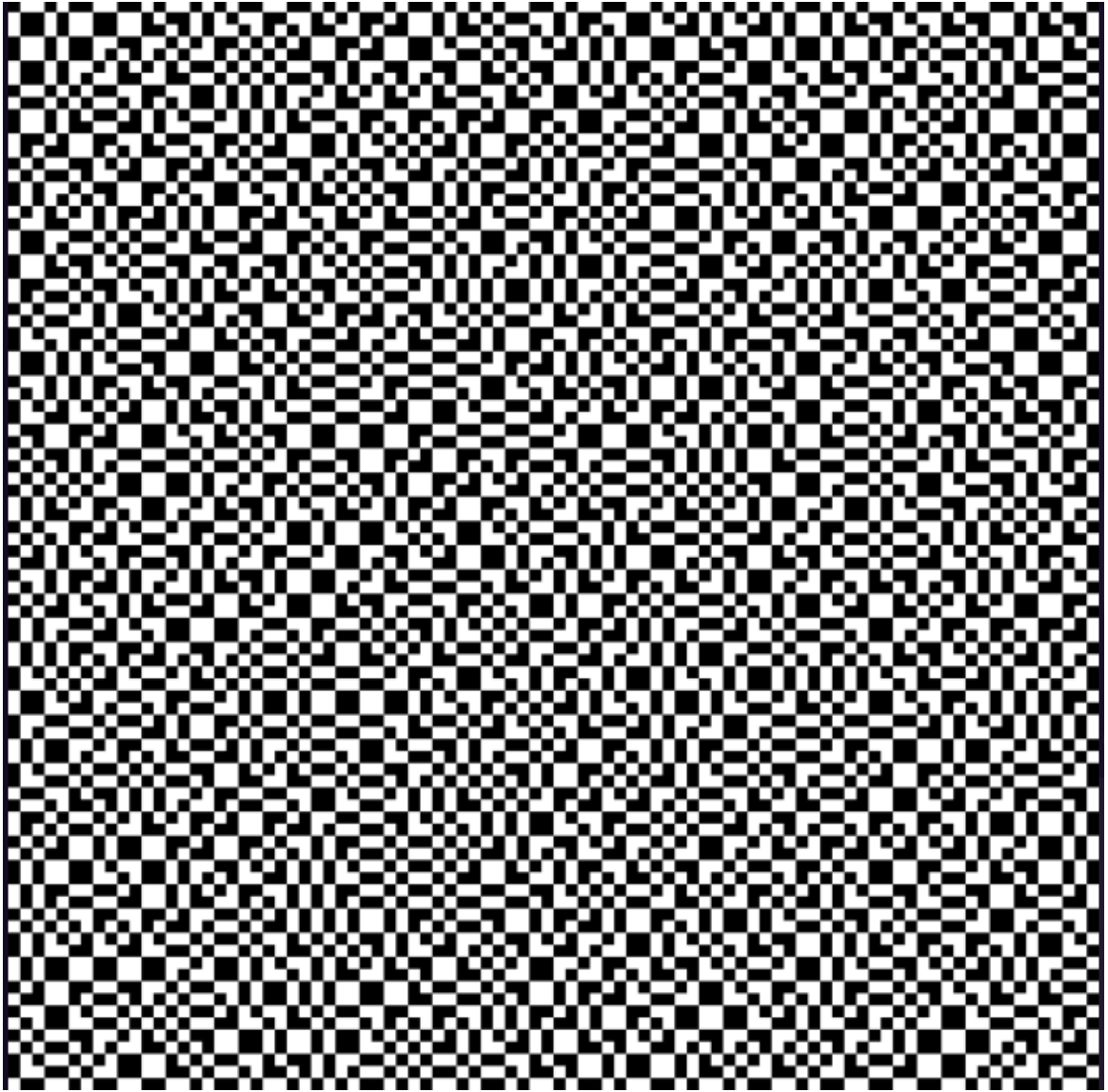
Berdasarkan pengujian bonus, sistem berhasil membentuk QR code dari recovery share yang dihasilkan pada saat pembuatan vault. QR code ini kemudian dibagi menjadi dua gambar share visual, di mana masing-masing gambar secara terpisah tidak cukup untuk memulihkan informasi asli. Setelah kedua gambar digabungkan kembali, sistem menghasilkan citra overlay yang dapat direkonstruksi menjadi QR code hasil pemulihan.

Ketika QR code hasil penggabungan dipindai, nilai yang diperoleh sama dengan recovery share awal. Hal ini menunjukkan bahwa mekanisme kriptografi visual pada implementasi bonus telah berjalan dengan benar dan dapat digunakan sebagai alternatif penyimpanan recovery share dalam bentuk visual. Walaupun fitur ini bersifat bonus, hasil pengujian menunjukkan bahwa integrasi fitur tersebut tetap konsisten dengan tujuan pemulihan vault pada sistem.

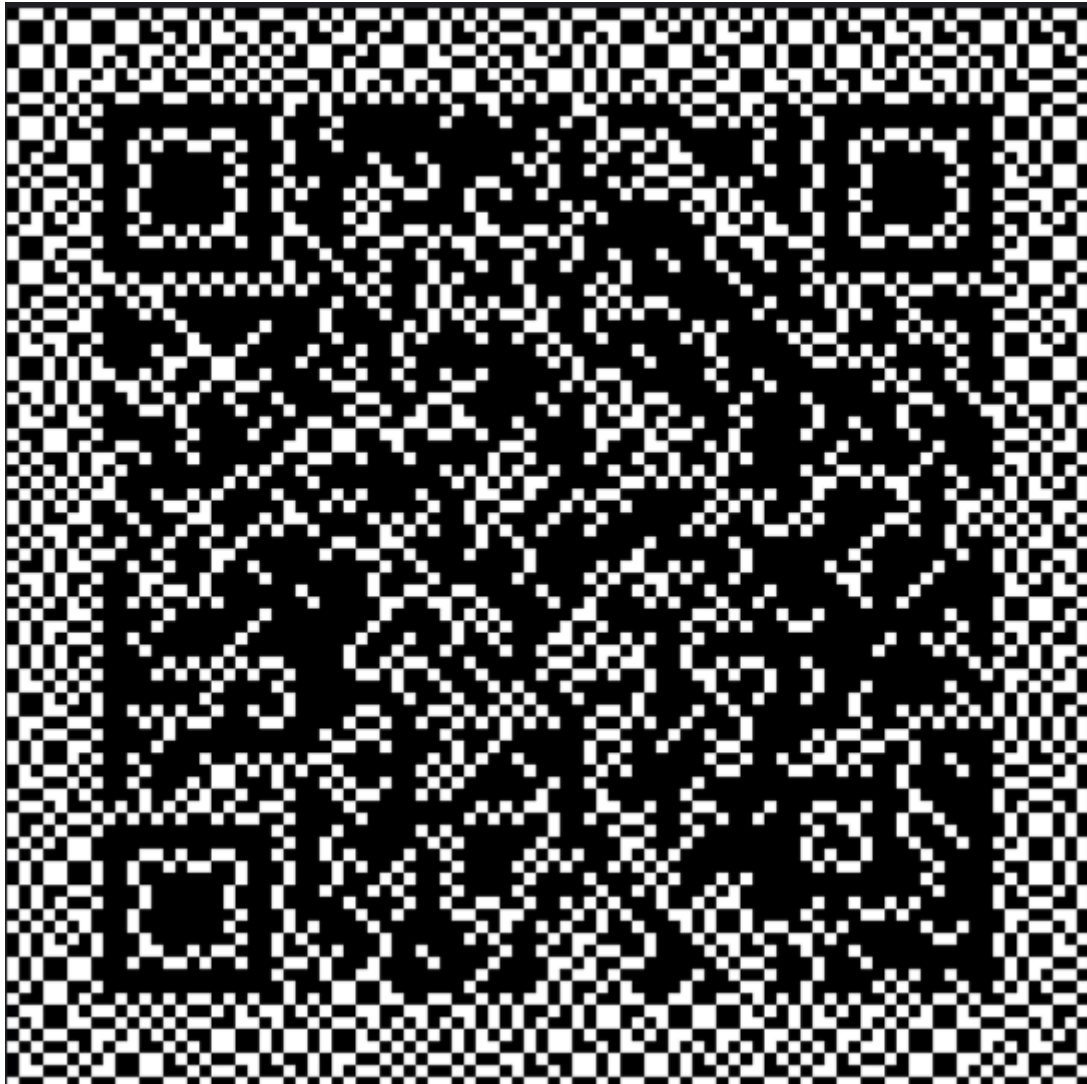


Gambar 3.9-a QR code awal dari recovery share





Gambar 3.9-b Dua visual share hasil pembagian QR code



Gambar 3.9-c Hasil penggabungan dua visual share

```
Welcome to Open vault
.....
Let's get started.
Upload both visual share images by providing their absolute file paths.
server: inactive | mode: backup | create vault disabled

1. Visual share image 1 absolute path*: /mnt/c/Users/Ghana/Documents/sss-vault/img1.png
> 2. Visual share image 2 absolute path*: /mnt/c/Users/Ghana/Documents/sss-vault/img2.png
3. Save
4. Cancel

up/down move | enter select | q back | ctrl+c quit
```

```
Welcome to Open vault
.....
Let's get started.
Upload both visual share images by providing their absolute file paths.
server: inactive | mode: backup | create vault disabled

  1. Visual share image 1 absolute path*: /mnt/c/Users/Ghana/Documents/sss-vault/img1.png
  2. Visual share image 2 absolute path*: /mnt/c/Users/Ghana/Documents/sss-vault/img2.png
> 3. Save
  4. Cancel

up/down move | enter select | q back | ctrl+c quit

.....
  1 function access() {
  2 * backup mode: local share + recovery share
  3 }
.....
> Press enter to continue: █
```

```
Welcome to nayaka vault (backup)
.....
Let's get started.
Backup mode is read-only.
server: inactive | mode: backup | create vault disabled

  1. View entry
> 2. Back

up/down move | enter select | q back | ctrl+c quit
```

Gambar 3.9-d Recovery share hasil pemindaian QR rekonstruksi

IV. KESIMPULAN

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, **Vault** berhasil dibangun sebagai aplikasi *password manager* terdistribusi berbasis CLI yang mengikuti ruang lingkup pada spesifikasi tugas. Sistem menggunakan arsitektur *client-server* dengan pembagian tanggung jawab yang cukup jelas, yaitu klien menangani proses kriptografi yang sensitif, sedangkan server berperan sebagai penyimpanan *server share*, *vault* terenkripsi, *nonce*, dan metadata pengguna di SQLite. Dengan pemisahan ini, server tidak menyimpan *master key* secara utuh, *local share*, *recovery share*, maupun isi *vault* dalam bentuk *plaintext*.

Pada aspek perlindungan data, isi *vault* dienkripsi sebagai satu kesatuan menggunakan AES-128-GCM, sementara *master key* dibangkitkan secara acak lalu dibagi menjadi tiga *share* menggunakan Shamir Secret Sharing dengan skema ambang batas (2,3). Implementasi mendukung dua mode akses sesuai spesifikasi, yaitu mode normal menggunakan kombinasi *local share* dan *server share*, serta mode backup menggunakan *local share* dan *recovery share*. *Local share* sendiri tidak disimpan dalam bentuk asli, tetapi dienkripsi menggunakan kunci turunan dari *master password* dengan Argon2id, sehingga data sensitif di sisi klien juga tetap memperoleh perlindungan tambahan.

Dari sisi fungsionalitas, sistem telah mendukung alur utama yang diminta, yaitu pembuatan *vault*, pembukaan *vault*, penambahan, pengubahan, dan penghapusan data *password* pada mode normal, serta akses baca pada mode backup. Setiap perubahan isi *vault* dilakukan di sisi klien, kemudian *vault* dienkripsi ulang dan diperbarui baik ke server maupun ke *backup* lokal terenkripsi. Selain itu, fitur pembangkitan *password* otomatis juga telah disediakan dengan memanfaatkan CSPRNG, sehingga aplikasi tidak hanya memenuhi kebutuhan penyimpanan *password*, tetapi juga membantu pengguna menghasilkan *password* yang lebih kuat.

Sebagai tambahan, implementasi bonus kriptografi visual juga telah diakomodasi dengan mengubah *recovery share* menjadi QR code dan membaginya menjadi dua gambar *share* yang dapat digabungkan kembali untuk memperoleh *recovery share* yang sama. Secara keseluruhan, hasil implementasi menunjukkan bahwa aplikasi ini telah memenuhi kebutuhan utama tugas, khususnya dalam menerapkan konsep *secret sharing*, enkripsi *vault*, pemisahan peran klien dan server, serta mekanisme akses normal dan pemulihan.

Meskipun demikian, keamanan sistem tetap bergantung pada pengelolaan *master password*, *local share*, dan *recovery share* oleh pengguna, sehingga aspek operasional tersebut tetap menjadi bagian penting dari penggunaan aplikasi ini.

DAFTAR PUSTAKA

1. Gunawan, E. E. (n.d.). *Pengembangan aplikasi manajemen kata sandi terdistribusi dengan skema pembagian data rahasia*. Institut Teknologi Bandung. <https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2025-2026/Makalah2025/Makalah-IF4020-Kriptografi-2025%20%2847%29.pdf>.
2. National Institute of Standards and Technology. (2023). *Advanced Encryption Standard (AES) (FIPS 197)*. <https://doi.org/10.6028/NIST.FIPS.197-upd1>.
3. Dworkin, M. (2007, November). *Recommendation for block cipher modes of operation: Galois/Counter mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-38D>.
4. Shamir, A. (1979). How to share a secret. *Communications of the ACM*, 22(11), 612-613. <https://doi.org/10.1145/359168.359176>.
5. Biryukov, A., Dinu, D., Khovratovich, D., & Josefsson, S. (2021, September). *Argon2 memory-hard function for password hashing and proof-of-work applications* (RFC 9106). RFC Editor. <https://doi.org/10.17487/RFC9106>.
6. Nir, Y., & Langley, A. (2018, June). *ChaCha20 and Poly1305 for IETF protocols* (RFC 8439). RFC Editor. <https://doi.org/10.17487/RFC8439>.
7. Naor, M., & Shamir, A. (1995). Visual cryptography. In A. De Santis (Ed.), *Advances in Cryptology - EUROCRYPT '94* (Lecture Notes in Computer Science, Vol. 950, pp. 1-12). Springer. <https://doi.org/10.1007/bfb0053419>.
8. ISO/IEC. (2024). *Information technology - Automatic identification and data capture techniques - QR code bar code symbology specification* (ISO/IEC 18004:2024). International Organization for Standardization. <https://www.iso.org/standard/83389.html>.

LAMPIRAN

Lampiran A Pranala Repository

Repository GitHub: <https://github.com/BP04/II4021-Tugas-4>

Lampiran B Pranala Video Demo

Video

Demo:

https://drive.google.com/drive/folders/1G9N-T4w5tQlwlNH9EabhyrS_unkXevtE?usp=sharing

Lampiran C Pembagian Tugas

Nama	NIM	Tugas
Benedict Presley	13523067	Server, models, config
Nayaka Ghana Subrata	13523090	Crypto, vault
Luckman Fakhmanidris Arvasirri	18223041	Client, tui